

SolarAgent Pro

Software Requirements Specification
& Project Initiation Document

Course Code: 0714 02 CSE 4202

Course: Web Programming & Mobile Application Development

WEEK 1 DELIVERABLE - INITIATION & SRS

Submitted By

Md. Sayem Hossian
Razu Sarder

Student ID 220217, 220220

Computer Science & Engineering Discipline, Khulna University

Course Code: 0714 02 CSE 4202

Course: Web Programming & Mobile Application Development

Submitted To

Dr. Kazi Masudul Alam

Professor,

Computer Science & Engineering Discipline, Khulna University

August 18, 2026

Table of Contents Error! Bookmark not defined.

1. Project Initiation	3
1.1 Introduction	3
1.2 Project Scope	3
1.3 Objectives	3
1.4 Roles & Responsibilities	4
1.5 Project Timeline Overview	4
1.6 AI-Assisted Development Methodology (AEDL)	4
2. Software Requirements Specification	6
2.1 Introduction	6
2.1.1 Purpose	6
2.1.2 Document Conventions	6
2.1.3 Intended Audience	6
2.1.4 Scope of Product	6
2.1.5 References	6
2.2 Overall Description	6
2.2.1 Product Perspective	6
2.2.2 Product Functions	7
2.2.3 User Classes and Characteristics	7
2.2.4 Operating Environment	7
2.2.5 Design and Implementation Constraints	7
2.2.6 Assumptions and Dependencies	8
2.3 Functional Requirements	9
2.4 Use Cases	10
2.5 External Interface Requirements	10
2.5.1 User Interfaces	10
2.5.2 Software Interfaces	10
2.5.3 Hardware Interfaces	10
2.5.4 Communication Interfaces	11
2.6 Non-Functional Requirements	11
2.7 Data Requirements	11
3. GitHub Repository & Contribution Setup	12
3.1 Repository Structure	12
3.2 Branching Strategy	12
3.3 Commit Message Convention	12
3.4 Contribution Guidelines Summary	12
3.5 Code Review Process	12
4. Initial Task List (Week 1 Output)	14
Appendix A: Glossary & Acronyms	15
Acknowledgement	15
Appendix B: Document Revision History	15

1. Project Initiation

1.1 Introduction

SolarAgent Pro is an AI-guided solar energy consultation system that takes a homeowner or shop owner from an initial expression of interest — “I want solar” — through to a complete, numerically grounded installation plan: site suitability, energy load, system sizing, exact components, a cost range, and a step-by-step installation roadmap. The system pairs five deterministic engineering calculation engines with a conversational AI agent, so that every number the user sees traces back to a specific, auditable calculation rather than to the language model’s own estimate.

This document records the Week 1 deliverables of the project’s initiation phase: scope, objectives, and roles; the Software Requirements Specification (Section 2); and the initial repository and task-list setup that Week 2 onward builds on (Sections 3–4).

1.2 Project Scope

The scope below reflects what has actually been implemented and verified as of this document’s date, not an aspirational target — every in-scope item was built, run, and checked against real inputs during development. Anything not yet integrated is listed explicitly as future work rather than left ambiguous.

IN SCOPE — SolarAgent Pro v1
✓ Site & irradiance suitability analysis
✓ Appliance load & peak-load calculation
✓ PV array, inverter & battery sizing
✓ Component recommendation (panels/inverter/battery/BOS)
✓ Installed-cost range & payback estimate
✓ Installation roadmap & safety guidance
✓ Conversational AI agent (tool-calling)
✓ Web UI: forms, results dashboard, chat panel
✓ Downloadable PDF / HTML report
✓ Roof-photo upload for visual context
OUT OF SCOPE (Future Work)
✗ Live weather API integration (PVGIS / Solcast)
✗ User authentication & multi-tenant accounts
✗ Persistent database (currently in-memory)
✗ Real-time local supplier pricing
✗ Native mobile app

Figure 1.1 — Project scope: implemented v1 functionality vs. future work

1.3 Objectives

- O1 - Reduce the expertise barrier to a preliminary solar system design from “requires a site engineer” to “a guided conversation,” without sacrificing calculation rigor.
- O2 - Ground every numeric output (energy, sizing, cost, payback) in an explicit, inspectable formula rather than an LLM estimate, so results are reproducible and auditable.
- O3 - Deliver a working full-stack reference implementation (FastAPI + React) that a developer can run locally within about 10 minutes of extracting the repository.
- O4 - Produce documentation and diagrams to a standard suitable for academic submission and for onboarding a new contributor without a walkthrough call.

- O5 - Establish a repeatable, AI-assisted development workflow (the “Agentic Engineering Development Lifecycle,” Section 1.6) that can be evaluated and refined across the remaining five weeks of the project.

1.4 Roles & Responsibilities

Table 1.1 below is shown as a diagram for readability; roles marked with a placeholder are included so this document is directly reusable for a team submission - fill in a name or remove the row if the project remains solo.



Figure 1.2 - Project roles and responsibilities

1.5 Project Timeline Overview

The six-week plan below is the one this document itself follows. Week 1 (this deliverable) is complete; Weeks 2–6 are planned and scoped in Section 1.6.

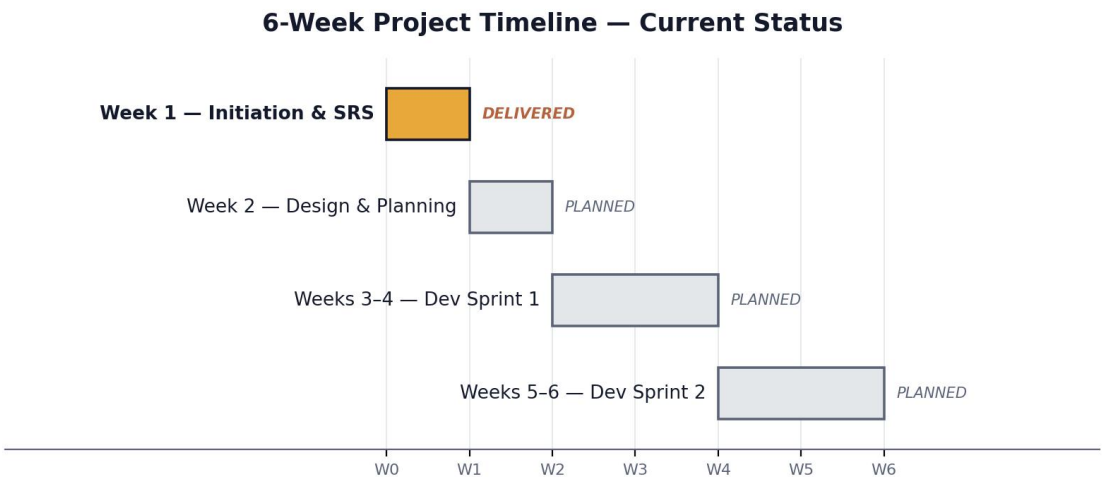


Figure 1.3 — Six-week project timeline and current status

1.6 AI-Assisted Development Methodology (AEDL)

This project follows a hybrid Agentic Engineering Development Lifecycle (AEDL): a standard six-week SDLC structure (initiation → design → two development sprints, a compressed Agile cadence) in which an AI assistant (Claude) is used deliberately at every stage — not merely for isolated code snippets, but for requirement drafting, full-stack implementation, test scaffolding, documentation, and sprint reporting — while the human developer

retains ownership of scope decisions, architectural review, and final acceptance. Table 1.2 maps each week's planned AI-assisted activity to this project's actual artifact.

Week	Planned AI-Assisted Activity	This Project's Artifact
1	AI-assisted requirement writing; AI generates README & task list	This SRS; README.md; docs/initial_task_list.md; CONTRIBUTING.md
2	AI design tools for wireframes; AI auto-generates architecture diagrams (ER, flowcharts)	Planned next — not yet started
3–4	AI (Copilot-style) code scaffolding; AI-generated unit tests & weekly progress summaries	Planned — core engines/agent/UI already implemented ahead of schedule; formal sprint framing pending
5–6	AI monitors sprint velocity, suggests backlog adjustments; AI-generated retrospectives	Planned — not yet started

Note: because the AEDL approach front-loaded a working implementation during scoping conversations, several Sprint 1 code deliverables (the five calculation engines, the tool-calling agent, and the React UI) already exist ahead of the Week 3–4 slot in which the original plan places them. Section 4 reconciles this by marking those backlog items “Done” rather than re-scheduling already-complete work.

2. Software Requirements Specification

2.1 Introduction

2.1.1 Purpose

This Software Requirements Specification defines the functional and non-functional requirements for SolarAgent Pro v1, as actually implemented in the backend (FastAPI, Python) and frontend (React, Vite, Tailwind) delivered alongside this document. It is written to IEEE 830-style conventions, adapted for a project of this scale, and is intended to serve both as a specification a reader can trace directly to the shipped code, and as the requirements baseline for Weeks 2 through 6.

2.1.2 Document Conventions

Functional requirements are numbered FR-1 through FR-10 and grouped by the engine or module that implements them. Non-functional requirements are numbered NFR-1 through NFR-6. Priority is expressed as High / Medium / Low, where High indicates the requirement is on the critical path for a usable end-to-end plan.

2.1.3 Intended Audience

- The project developer - as a design and implementation reference.
- A course instructor or evaluator - for requirements traceability against the delivered system.
- Any future contributor - extending the system in Weeks 2–6 and beyond.

2.1.4 Scope of Product

See Section 1.2 and Figure 1.1.

2.1.5 References

- IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications.
- Anthropic, Claude API documentation - platform.claude.com/docs
- Project repository - solaragent-pro (see README.md for setup).

2.2 Overall Description

2.2.1 Product Perspective

SolarAgent Pro is a new, standalone system - it does not replace or integrate with an existing product. It depends on exactly one external service for its conversational layer (the Anthropic Claude API) and is designed so that a second external dependency (a live weather/irradiance API) can be added later without changing any other component, as shown in Figure 2.1.

System Context Diagram (Level 0)

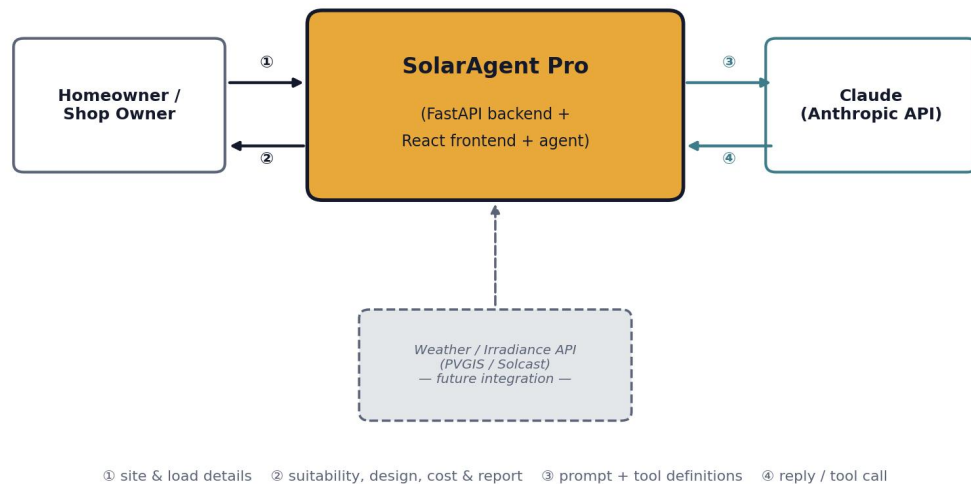


Figure 2.1 — System context diagram (Level 0)

2.2.2 Product Functions

- Estimate site solar resource and score installation suitability.
- Calculate daily energy consumption and peak load from an appliance list.
- Size a PV array and, where applicable, a battery bank.
- Recommend specific components (panels, inverter, battery, mounting, BOS).
- Estimate installed cost range and payback period.
- Generate an installation roadmap with safety and maintenance guidance.
- Provide a conversational agent that explains and answers questions about the above.
- Generate a downloadable report.

2.2.3 User Classes and Characteristics

Primary user class: a homeowner or small-shop owner considering solar, with no assumed technical or electrical-engineering background. The system is designed to be usable end-to-end by this class without external help, per Objective O1. No secondary or administrative user class exists in v1 (see NFR-3 and the Out-of-Scope items in Figure 1.1 regarding authentication).

2.2.4 Operating Environment

Backend: any environment capable of running Python 3.10+ and Uvicorn. Frontend: any modern evergreen browser, served as a static Vite build. There is no mobile-native runtime; the web UI is responsive down to mobile viewport widths.

2.2.5 Design and Implementation Constraints

- Must use the Anthropic Claude API (Messages API, Python SDK) for the conversational layer.
- Numeric outputs must derive from deterministic Python functions, never from unvalidated model output — a self-imposed constraint driving the tool-calling architecture (see FR-7).
- No cloud-vendor-specific services, to keep the reference implementation portable (NFR-6).

2.2.6 Assumptions and Dependencies

- Assumes the operator supplies a valid Anthropic API key at deployment time.
- Assumes the built-in irradiance/tariff estimation model is acceptable for preliminary planning; final designs are assumed to be validated by a licensed electrician/engineer against local code (this is stated explicitly to the end user in-product).
- Assumes single-instance deployment for v1 (in-memory session store; see NFR-2).

2.3 Functional Requirements

ID	Requirement	Priority	Source Module
FR-1	Accept country, city, roof type, roof area, shading, and building type; estimate peak sun-hours, average temperature and tariff; compute a 0–100 suitability score with contributing factors.	High	engines/site_analysis.py
FR-2	Accept an appliance list (name, power, hours/day, quantity, simultaneity flag) and compute daily energy (kWh/day) and estimated peak load (W) using a diversity factor.	High	engines/energy_engine.py
FR-3	Compute required PV array size (kW) via the peak-sun-hour method, and required battery capacity (kWh) via depth-of-discharge and autonomy-day targets for hybrid/off-grid modes.	High	engines/energy_engine.py
FR-4	Recommend panel tier/count/wattage, inverter type/size, battery chemistry/voltage/capacity, mounting structure, and balance-of-system items.	High	engines/component_engine.py
FR-5	Compute installed-cost range, estimated annual generation and savings, and payback period.	High	engines/cost_engine.py
FR-6	Generate a phased installation roadmap, safety notes, common mistakes, and a maintenance schedule, adapted to the selected system mode.	Medium	engines/roadmap_engine.py
FR-7	Provide a chat interface backed by an LLM agent that calls FR-1–FR-6 as tools, explains results in plain and technical language, asks clarifying questions, and never states a figure not derived from a tool call.	High	agent.py, system_prompt.py
FR-8	Generate a downloadable report (PDF via backend, HTML via live demo) summarizing site analysis, design, cost, and roadmap.	Medium	report_generator.py
FR-9	Allow an optional roof-photo upload, passed to the AI agent as visual context.	Low	frontend + agent.py
FR-10	Support toggling between dark and light visual themes.	Low	frontend

2.4 Use Cases

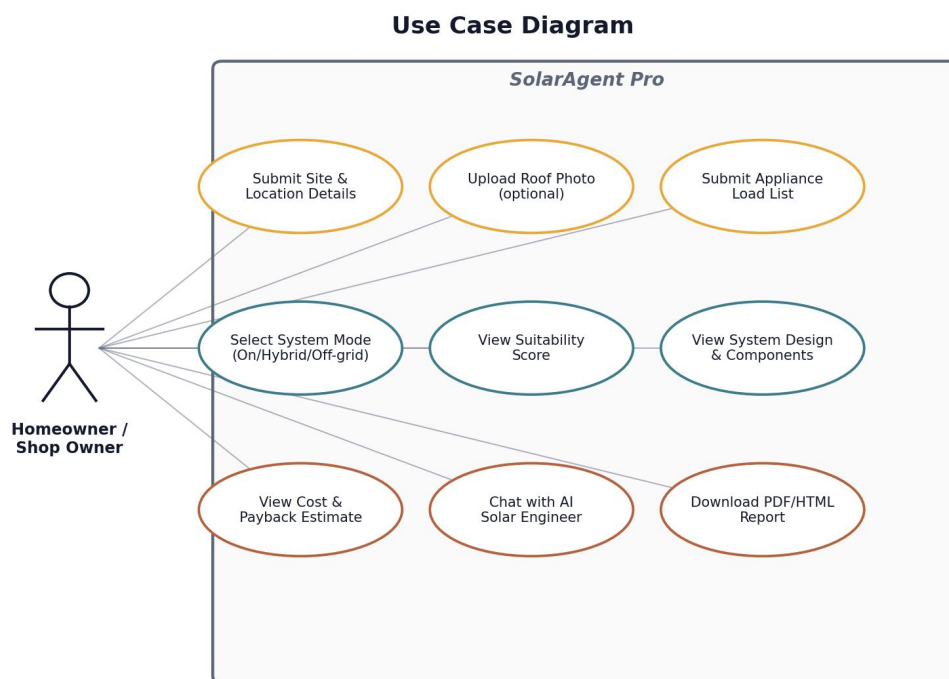


Figure 2.2 — Use case diagram

ID	Name	Precondition	Main Flow	Postcondition
UC-1	Submit Site Details	Session created	User enters country, city, roof type, area, shading, building type and submits.	Suitability + resource estimate available
UC-2	Submit Appliance Load	Site details submitted	User adds appliances (presets or manual), selects system mode & autonomy days.	Full design, cost, roadmap computed
UC-3	View Results Dashboard	Load submitted	System displays suitability gauge, system design, cost, and roadmap.	User has a complete plan
UC-4	Chat with AI Engineer	Session exists (site/load optional)	User asks a question; agent calls relevant tools and/or grounds its answer in existing session data.	User receives a grounded explanation
UC-5	Download Report	Design computed	User requests the report; system generates a PDF (backend) or HTML (live demo).	User has a downloadable file

2.5 External Interface Requirements

2.5.1 User Interfaces

A responsive web UI (React): structured forms for site and load input, a results dashboard, and a chat panel, with dark/light theming.

2.5.2 Software Interfaces

The Anthropic Messages API (HTTPS/JSON) via the official Python SDK, with the model configurable through an environment variable. Frontend-to-backend communication is REST/JSON over HTTP with CORS enabled for the configured origin.

2.5.3 Hardware Interfaces

None specific — a standard client device capable of running a modern browser.

2.5.4 Communication Interfaces

HTTPS to the Anthropic API; HTTP between frontend and backend in local development (HTTPS recommended for any real deployment).

2.6 Non-Functional Requirements

ID	Category	Requirement
NFR-1	Performance	Deterministic engine calculations complete in well under 100 ms; conversational turns are bounded by LLM API latency plus a hard cap of 6 tool-loop iterations to prevent runaway calls.
NFR-2	Reliability	All five engines are pure functions (identical input → identical output), giving reproducible results. The current session store is in-memory only and does not survive a backend restart flagged for hardening in a later sprint.
NFR-3	Security	No authentication is implemented in v1. The Anthropic API key lives server-side only and is never exposed to the frontend. CORS is restricted to configured origins.
NFR-4	Usability	Responsive layout for mobile and desktop. Every suitability score is shown with its contributing factors, never as a bare number. The agent's system prompt requires plain-language explanations alongside technical detail.
NFR-5	Maintainability	Each engine is isolated with no cross-dependencies beyond explicit function parameters e.g. the irradiance model can be replaced with a live weather API without touching sizing, cost, or roadmap logic.
NFR-6	Portability	Backend is a standard FastAPI/Uvicorn service; frontend is a standard Vite/React static build. Neither depends on a specific cloud vendor.

2.7 Data Requirements

Session state (site input, computed site analysis, load input, energy/design/cost/roadmap results, and conversation history) is held per session in the backend's SessionStore. No personally identifying information is collected beyond what the user voluntarily enters into the site/load forms or chat, and no data is persisted beyond process memory in v1 (see NFR-2 and the Out-of-Scope items in Figure 1.1).

3. GitHub Repository & Contribution Setup

3.1 Repository Structure

The repository is organized as two independently runnable applications sharing one methodology, plus documentation and a standalone demo:

- backend/ - FastAPI service: app/main.py (routes), app/agent.py (tool-calling agent), app/engines/ (five calculation engines), app/report_generator.py (PDF).
- frontend/ - React + Vite + Tailwind: src/components/ (forms, dashboard, chat), src/context/ (session state), src/api.js (backend client).
- docs/ - example_conversations.md, sample_report.md, initial_task_list.md, and this SRS.
- live-demo/ - a self-contained single-file version for zero-setup evaluation.
- .github/ - pull request template.

3.2 Branching Strategy

This project uses GitHub Flow rather than full GitFlow, since a six-week project of this size does not benefit from GitFlow's release/hotfix branch overhead:

- main is always in a runnable state.
- New work happens on feature/<short-description> or fix/<short-description> branches cut from main.
- Work merges back to main via Pull Request, even for a solo contributor — this preserves the PR description and review-comment trail as part of the project's documentation.

3.3 Commit Message Convention

Commits follow the Conventional Commits format — <type>: <short imperative description> — so the history itself is a readable changelog:

- feat: - a new capability (e.g. "feat: add battery sizing to energy engine")
- fix: - a bug fix
- docs: - documentation only
- refactor: - code change with no behavior change
- test: - adding or updating tests
- chore: - tooling, dependencies, config

3.4 Contribution Guidelines Summary

Full guidelines live in CONTRIBUTING.md at the repository root. In summary: branch from main, keep PRs scoped to one logical change, run the relevant app locally before opening a PR (see README.md "Quick start"), and fill in the PR template's checklist.

3.5 Code Review Process

Every PR is reviewed against a short checklist before merge: does it run locally as described; are new environment variables documented in the relevant .env.example; if behavior changed, was README.md or this SRS updated to match; and - specific to this project's architecture - does any new numeric output trace to an engine function rather than to unvalidated model output (per FR-7 and the constraint in Section 2.2.5).

4. Initial Task List (Week 1 Output)

This is the seed backlog Week 2 will load into a GitHub Projects board. It is intentionally an epic-level summary; the granular, board-ready version lives in docs/initial_task_list.md so it can be copied directly into GitHub Issues.

Epic	Task	Target	Priority	Status
E1	Site analysis engine	Wk 1–2	High	Done
E2	Load & sizing engine	Wk 1–2	High	Done
E3	Component & cost engines	Wk 3–4	High	Done
E4	Tool-calling conversational agent	Wk 3–4	High	Done
E5	React frontend (forms, dashboard, chat)	Wk 3–4	High	Done
E6	PDF/HTML report generation	Wk 5–6	Medium	Done
E7	UI wireframes & architecture diagrams	Wk 2	Medium	Not started
E8	GitHub Projects board configuration	Wk 2	Medium	Not started
E9	Authentication & persistent session store	Wk 5–6	Medium	Not started
E10	Live weather API integration	Post-Wk 6	Low	Not started
E11	Automated test suite for engines	Wk 5–6	Medium	Not started

Appendix A: Glossary & Acronyms

Term	Definition
SRS	Software Requirements Specification.
PSH	Peak Sun-Hours — the number of hours per day at which solar irradiance would need to be 1,000 W/m ² to deliver the day's actual total irradiation.
DoD	Depth of Discharge — the fraction of a battery's capacity usable before recharging.
BOS	Balance of System — every component in an installation other than panels, inverter, and battery (cabling, protection devices, mounting, etc.).
MPPT	Maximum Power Point Tracking — a charge-controller technique that maximizes power draw from a PV array.
LiFePO4	Lithium Iron Phosphate — a lithium battery chemistry favored for solar storage for its safety and cycle life.
kW / kWp	Kilowatt / kilowatt-peak — unit of PV array rated capacity.
BOM	Bill of Materials.
REST	Representational State Transfer — the API architecture style used by the backend.
CORS	Cross-Origin Resource Sharing.

Acknowledgement

This project was completed with the support of several tools and technologies. Each is acknowledged briefly below, along with the reason it was chosen.

- Claude (Anthropic) - used for AI-assisted requirement drafting, code generation, and documentation throughout this project (Section 1.6), chosen for its tool-calling capability, which lets the agent ground every answer in a real calculation rather than a guess.
- GitHub - used for version control and the pull-request review workflow described in CONTRIBUTING.md, chosen for its ubiquity and traceable change history.
- FastAPI (Python) - used for the backend API, chosen for built-in request validation and automatic interactive API documentation.
- React, Vite & Tailwind CSS - used for the frontend, chosen for fast component-based development and consistent design-system styling.
- Anthropic Claude API (Messages API) - powers the conversational agent itself, distinct from Claude's use as a development assistant above, chosen specifically for its structured tool-use support (FR-7).
- matplotlib & docx (Node.js) - used to generate this document and its diagrams as code, chosen to keep them reproducible and easy to update as the project changes.
- reportlab (Python) - used for the in-app PDF report feature, chosen for lightweight PDF generation without a browser-rendering dependency.

Appendix B: Document Revision History

Version	Date	Author	Change
1.0	August 17, 2026	Razu Sarder/Md. Sayem Hossain / Claude (AI-assisted)	Initial Week 1 SRS & Project Initiation Document.